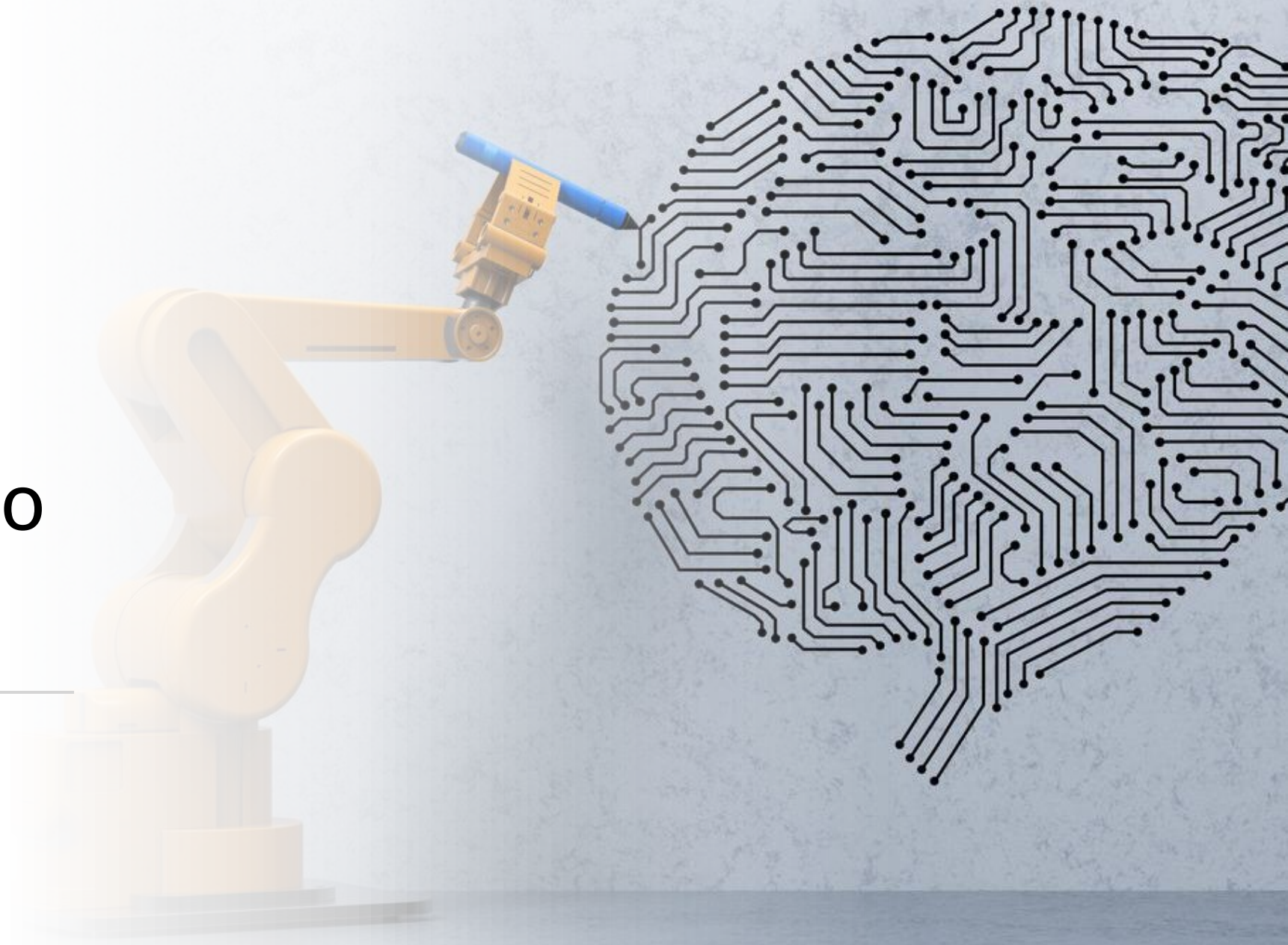
Reconocimiento de Patrones

Clase 14

Arboles de decisión y Bosques Aleatorios



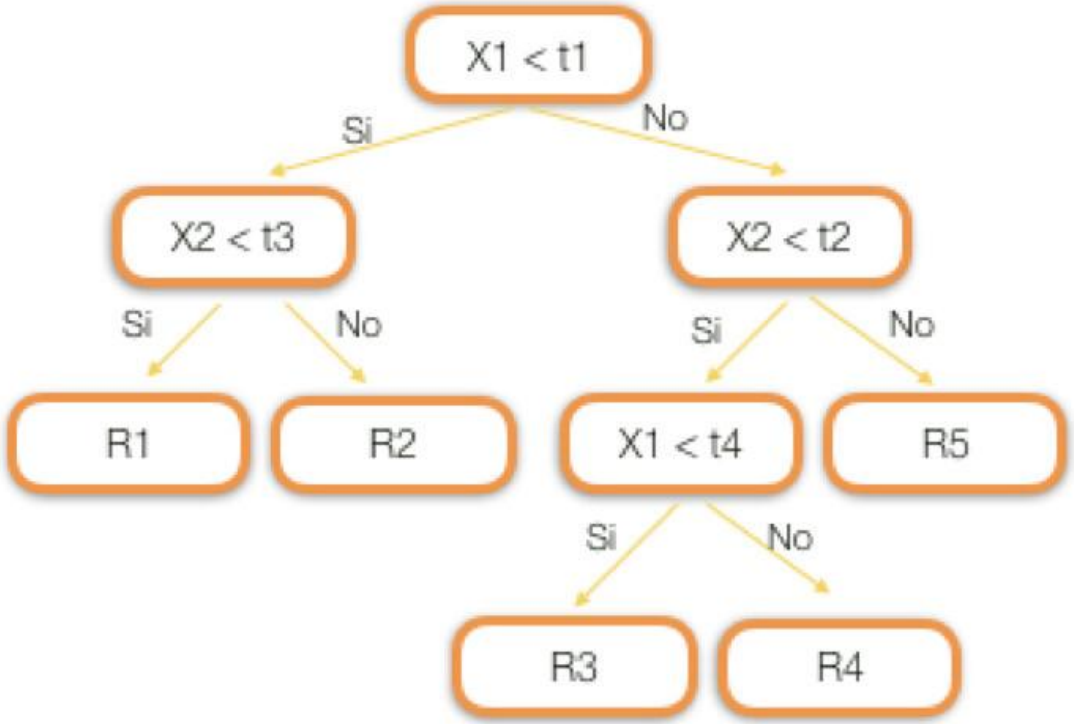
Antes de empezar

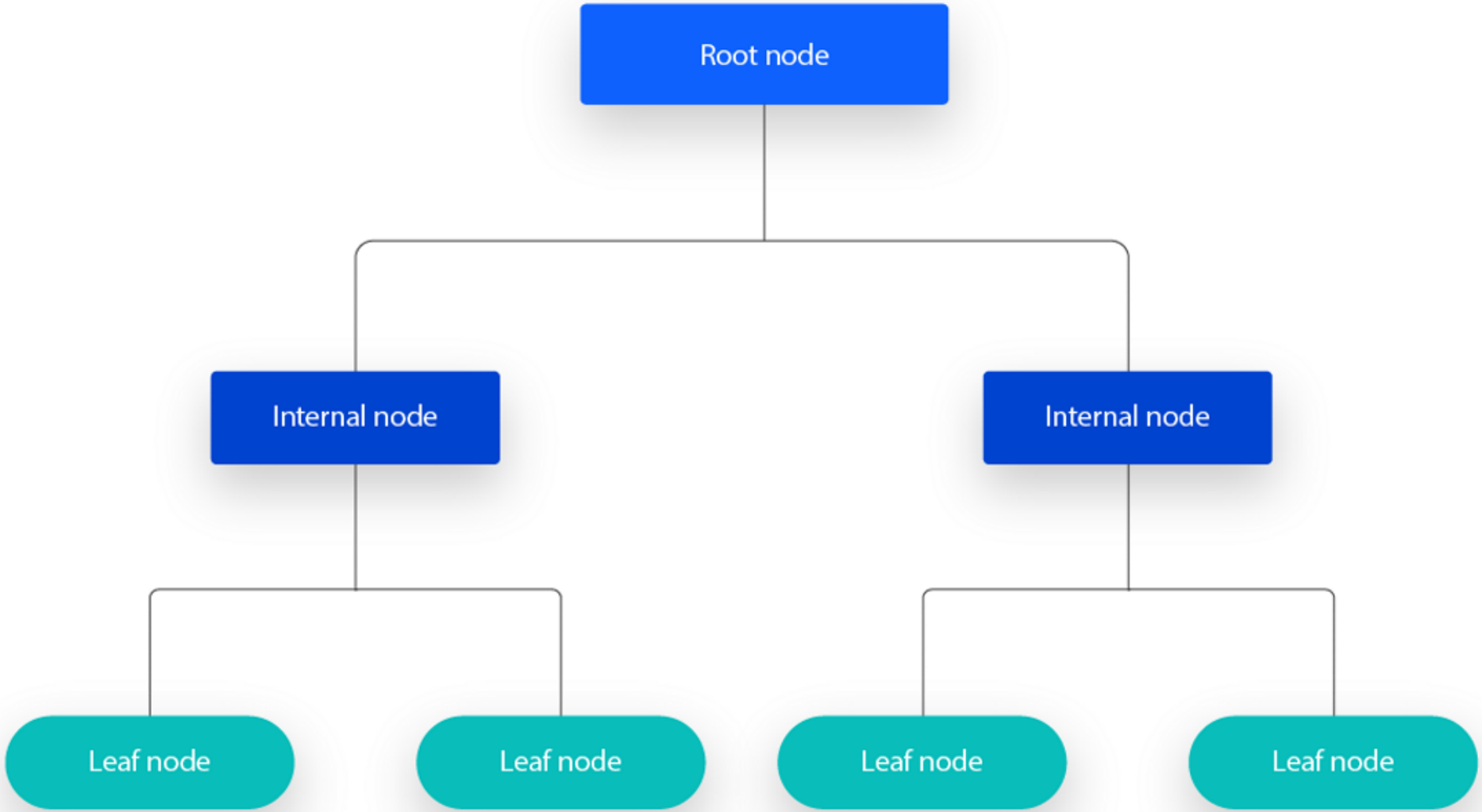
- ¿Dudas de la clase anterior?



Arbol de decisión

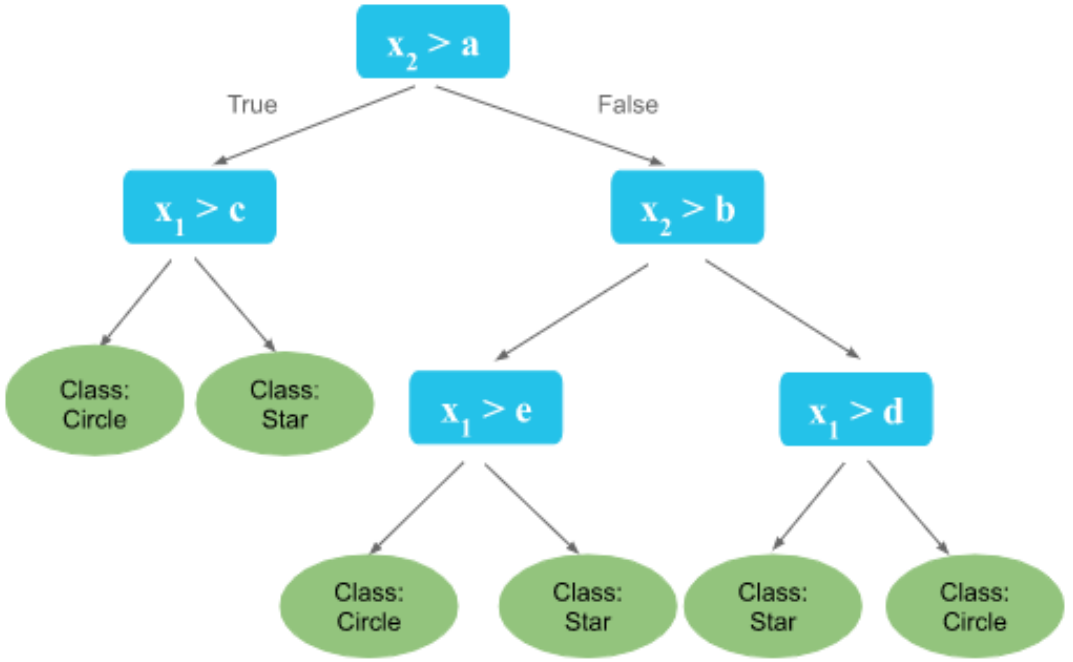
- Es un modelo supervisado que divide el espacio de características en regiones usando reglas del tipo “si…entonces…”
- El algoritmo busca dividir los datos de manera que cada grupo sea lo más “puro” posible.
- Este está compuesto por un nodo raíz, ramas, nodos internos y hojas.



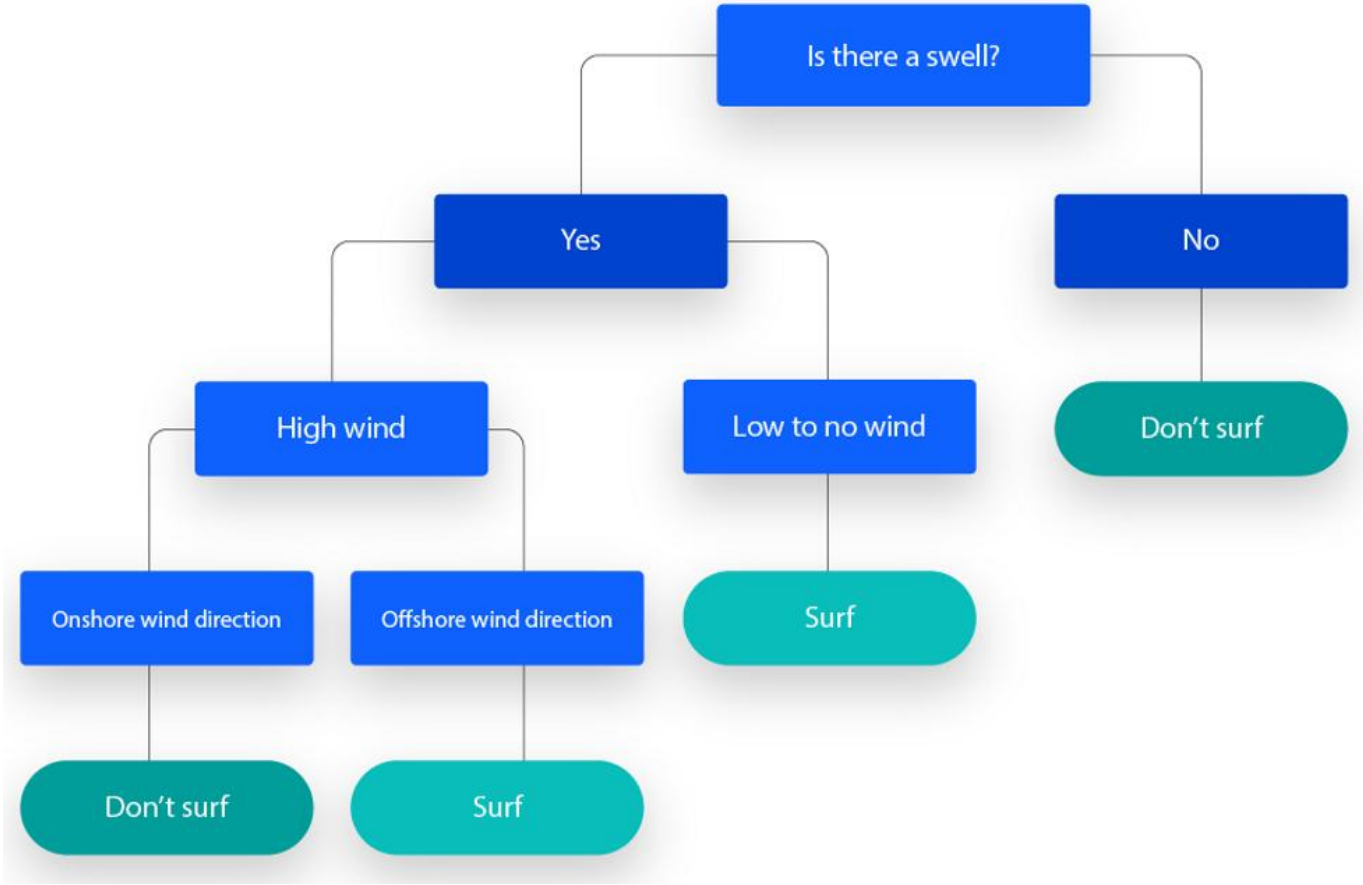

Root node
Internal node
Internal node
Leaf node
Leaf node
Leaf node
Leaf node

Clasification And Regresion Trees (CART)

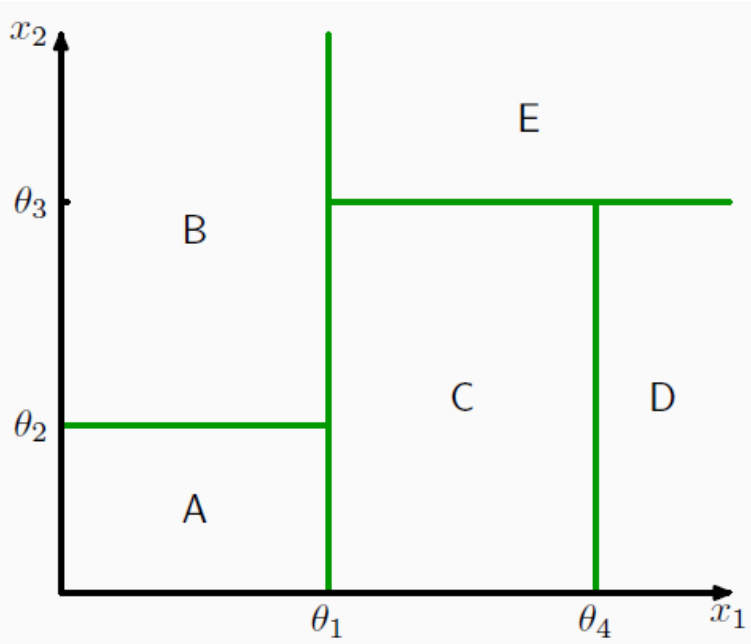
- Este es uno de los algoritmos más importantes para construir arboles de decisión, el cual siempre genera **árboles binarios**: cada nodo se divide en exactamente **dos ramas** (sí/no), nunca más.
- El espacio se divide de forma recursiva mediante un conjunto anidado de reglas.

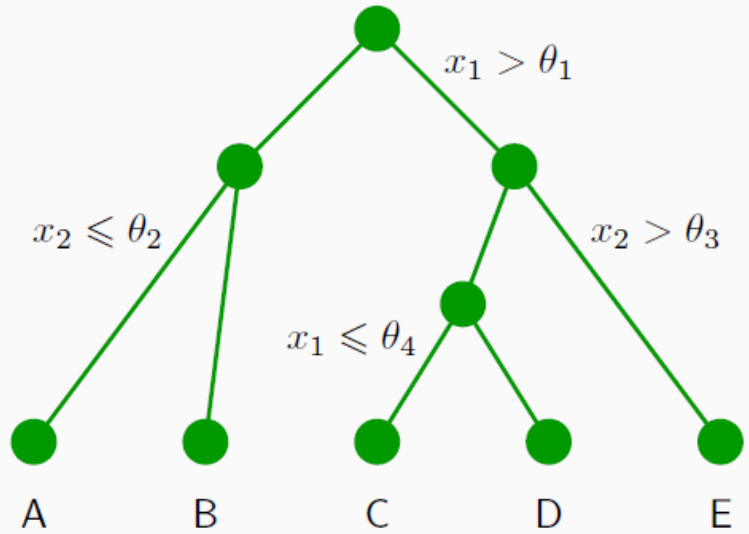


- Nodo raíz: es el punto de partida del árbol, es la primera pregunta que hace el modelo y representa el atributo más informativo.
- Nodo interno: son lo puntos donde se evalúa una condición (pregunta tipo "sí/no") sobre una característica predictora específica para subdividir los datos.
- Hoja: Son los resultados finales del árbol, contienen la predicción (clase) para los datos que llegan a ellos.

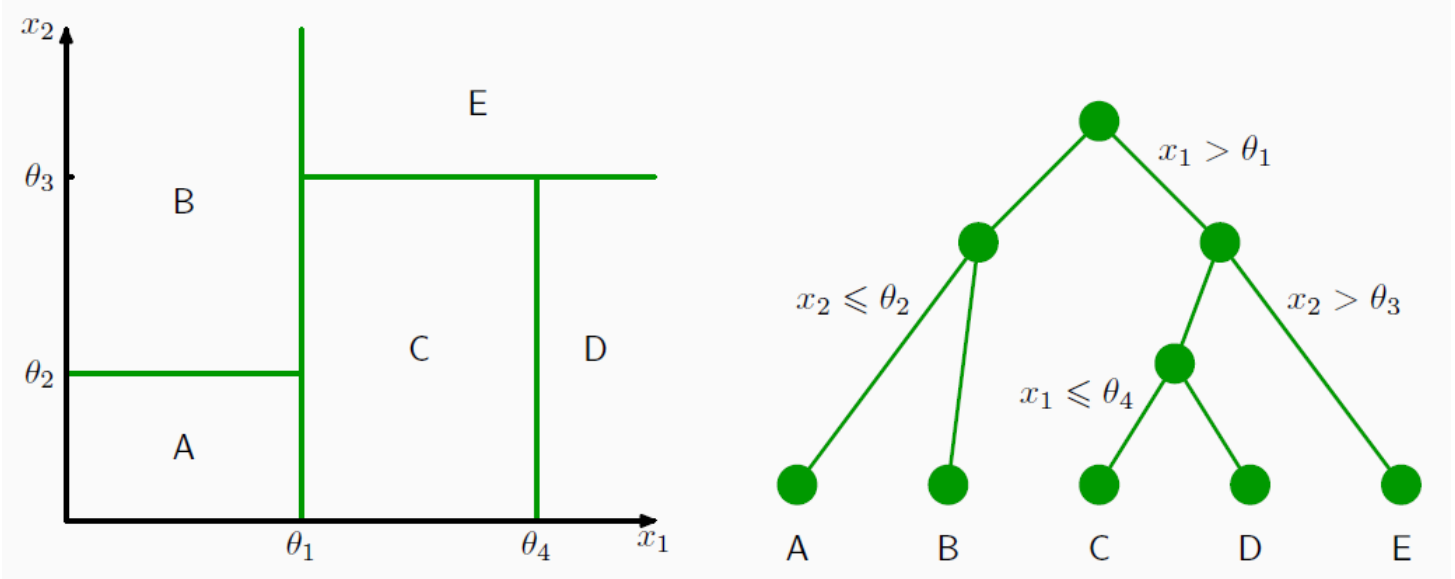


- Dado un conjunto de datos $\{x^{(1)}, \cdots, x^{(n)}\} \in R^2$ cada división tiene la forma $x_j < t$, lo cual genera una división del espacio en regiones rectangulares (hiper rectángulos).





¿Cómo construir el árbol?



Para aprender un árbol de decisión es necesario determinar:

- Qué valor se asocia a cada región
- Qué atributo se elige en cada nodo y qué regla se aplica a dicho atributo

Proceso general de construcción

- El algoritmo sigue una estrategia voraz (greedy) que consta de las siguientes partes:

1. Calcular la impureza del nodo actual.
2. Para cada variable y cada posible umbral, calcular la ganancia de impureza del split.
3. Aplicar el split con menor impureza.
4. Repetir recursivamente en los **dos** nodos hijo resultantes.
5. Detener cuando se cumple algún criterio de parada.

Medidas de Impureza

- Entropía

Es una medida de la impureza, desorden o incertidumbre en un conjunto de datos y esta dada por:

$$H(S) = -\sum_{i=1}^{c} p_i log_2(p_i)$$

Donde p_i es la proporción de ejemplos de la clase i. Cuando todos los ejemplos son de una misma clase H=0, representando una máxima pureza, pero si las clases están mezcladas H=1, significando una máxima impureza

- Índice de Gini

Mide la probabilidad de clasificar incorrectamente un ejemplo elegido aleatoriamente si se etiqueta según la distribución de clases del nodo. Es la medida utilizada por CART por defecto.

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$$

Donde p_i es la proporción de ejemplos de la clase i. Para dos clases, Gini=0 cuando el nodo es completamente puro, y Gini=0.5 cuando hay una máxima impureza (la misma proporción de clases).

Ejemplo

Supongamos un nodo con 10 ejemplos: 7 de la clase A y 3 de la clase B.

Las proporciones son p(A) = 7/10 = 0.7 y p(B) = 3/10 = 0.3.

- Entropía

$$H(S) = -\sum_{i=1}^{c} p_i log_2(p_i)$$

H = − [0.7 · $\log_2(0.7)$ + 0.3 · $\log_2(0.3)$]

= − [0.7 · (−0.515) + 0.3 · (−1.737)]

= − [−0.360 + (−0.521)]

= − [−0.881]

= 0.881

Ejemplo

p(A) = 7/10 = 0.7

p(B) = 3/10 = 0.3.

- Gini

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$$

Gini = $1 - (0.7^2 + 0.3^2)$

= $1 - (0.49 + 0.09)$

= $1 - 0.58$

= 0.420

H(S) = 0.881
Gini(S) = 0.420

Por lo tanto, el nodo es bastante impuro

Ganancia de Gini y selección de split

- Para elegir la mejor división, CART calcula el **Gini ponderado** de los dos nodos hijos:

$$Gini_{split} = \frac{N_L}{N} Gini(S_L) + \frac{N_R}{N} Gini(S_R)$$

Donde:

N_L : numero de ejemplos que van al nodo izquierdo

N_R : numero de ejemplos que van al nodo derecho

N: numero de ejemplos que había originalmente

S_L y S_R : Subconjuntos izquierdo y derecho

Entonces, para saber cual es al mejor división el algoritmo hace una búsqueda exhaustiva

Para cada atributo disponible:
  Para cada valor posible de ese atributo:
    Dividir datos en dos grupos (≤ valor y > valor)
    Calcular Gini de esa división
    Guardar si es la mejor hasta ahora
Ejecutar la división ganadora

Datos: [22, 25, 28, 35, 40, 45, 55, 60]

CART prueba cortes en:
¿Edad ≤ 23.5? → Gini = 0.45
¿Edad ≤ 26.5? → Gini = 0.38
¿Edad ≤ 31.5? → Gini = 0.21 ← ✅ MEJOR
¿Edad ≤ 37.5? → Gini = 0.33
...

- Por ejemplo, supongamos que en nuestro nodo tenemos los siguientes ejemplos:

Edad	Ingreso	¿Compra producto?
30	20000	Sí
40	50000	Sí
20	30000	No
50	60000	No
60	80000	Sí

Supongamos que queremos dividir los datos basados en la edad con un umbral de 35 años, el nodo hijo izquierdo tendrán los datos que tienen una edad menor o igual a 35, mientras que el nodo hijo derecho tendrá los datos con una edad mayor a 35 años.

- Izquierdo (edad <= 35)

Edad	Ingreso	¿Compra producto?
30	20000	Sí
20	30000	No

Impureza Gini

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2 = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.5$$

Ambos son igualmente probables, es decir, tienen la misma proporción de datos. Al ser de 0.5 se entiende que es una nodo impuro.

- Derecho (edad >35)

Edad	Ingreso	¿Compra producto?
40	50000	Sí
50	60000	No
60	80000	Sí

Impureza Gini

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2 = 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = 0.444$$

Al tener un Gini de 0.444 se pude considerar que el nodo es impuro ya que esta más cerca del 0.5 que del 0.

- Para saber que tan bien se realizo el Split con el umbral y característica elegida, se evalúa el Gini ponderado

$$Gini_{split} = \frac{N_L}{N} Gini(S_L) + \frac{N_R}{N} Gini(S_R)$$

$$Gini_{split} = \frac{2}{5}(0.5) + \frac{3}{5}(0.444) = 0.466$$

El algoritmo probara diferentes características y umbrales para encontrar el Split que minimiza la impureza Gini.

Evaluación de variables continuas y categóricas

- Variables continuas

Ordenar los valores de la feature x_j, probar thresholds entre puntos consecutivos:

$$t = \frac{x_{(i)} + x_{(i+1)}}{2}$$

Para cada valor de t, dividir en S_L y S_R, se calcula el Gini ponderado y de todas las posibles divisiones se selecciona la mejor opción.

- Variables categóricas binarias

Como solo hay un Split posible, se establece '¿Valor = categoría_A?' y se calcula la ganancia directamente.

- Variables categóricas con múltiples valores

Para k categorías, se prueba todas las particiones binarias.

Ejemplo

Dataset: 8 pacientes.
Variable objetivo:
¿Tiene la enfermedad?
(Sí / No)

Distribución inicial:

4 Sí, 4 No →

Gini(raíz) = $1 - (0.5^2 + 0.5^2)$ = **0.500**

#	Fiebre	Tos	Dolor de cabeza	Enfermedad
1	Alta	Sí	Sí	Sí
2	Alta	Sí	No	Sí
3	Baja	No	Sí	No
4	Baja	Sí	No	No
5	Alta	No	Sí	Sí
6	Baja	No	No	No
7	Alta	No	No	No
8	Baja	Sí	Sí	Sí

¿Cuál es el mejor split?

- Evaluamos cada variable calculando la Ganancia de Gini = Gini(padre) − Gini ponderado(hijos).

Variable: Fiebre

Rama	Pacientes	Sí	No	Gini
Alta (1,2,5,7)	4	3	1	$1-(0.75^2+0.25^2)$ = **0.375**
Baja (3,4,6,8)	4	1	3	$1-(0.25^2+0.75^2)$ = **0.375**

Gini ponderado = (4/8)×0.375 + (4/8)×0.375 = 0.375

ΔGini(Fiebre) = 0.500 − 0.375 = 0.125

Variable: Tos

Rama	Pacientes	Sí	No	Gini
Alta (1,2,4,8)	4	3	1	$1-(0.75^2+0.25^2) = \mathbf{0.375}$
Baja (3,5,6,7)	4	1	3	$1-(0.25^2+0.75^2) = \mathbf{0.375}$

Gini ponderado = 0.375

ΔGini(Tos) = 0.500 − 0.375 = 0.125

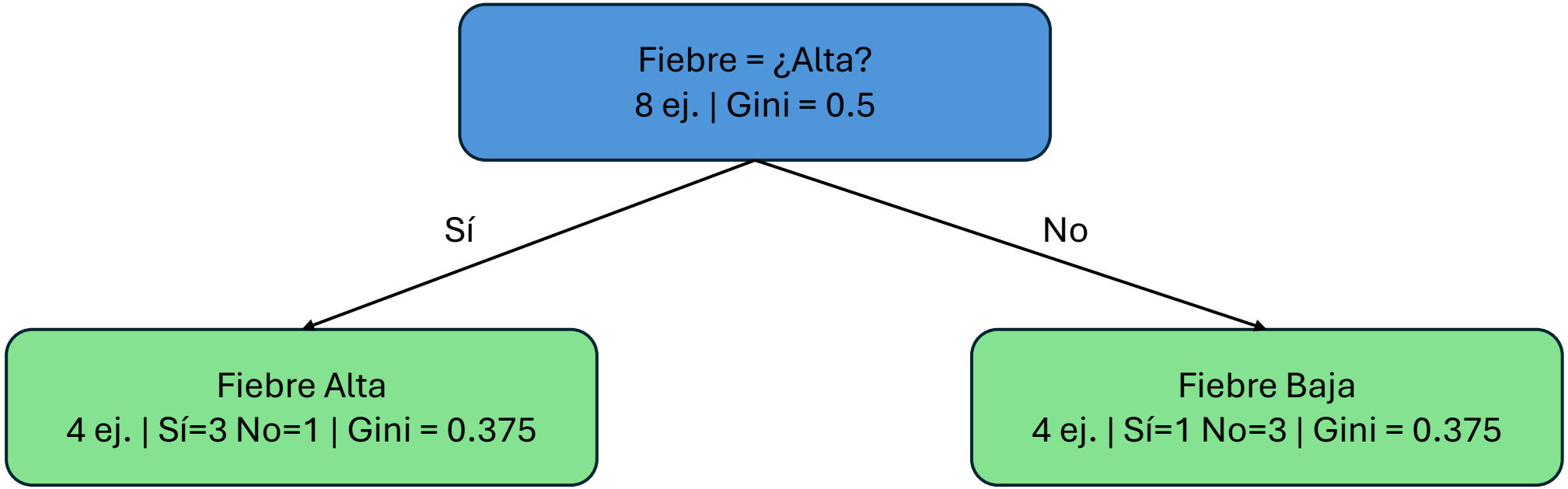
Variable: Dolor de cabeza

Rama	Pacientes	Sí	No	Gini
Alta (1,3,5,8)	4	3	1	$1-(0.75^2+0.25^2) = \mathbf{0.375}$
Baja (2,4,6,7)	4	1	3	$1-(0.25^2+0.75^2) = \mathbf{0.375}$

Gini ponderado = 0.375

ΔGini(Dolor) = 0.500 − 0.375 = 0.125

Todas tienen la misma ganancia (0.125). El algoritmo elige la **primera encontrada: Fiebre**.



Nivel 2, rama izquierda: Fiebre Alta

Distribución: Sí=3 (#1,2,5), No=1 (#7). Gini = 0.375.

#	Fiebre	Tos	Dolor de cabeza	Enfermedad
1	Alta	Sí	Sí	Sí
2	Alta	Sí	No	Sí
5	Alta	No	Sí	Sí
7	Alta	No	No	No

Variable: Tos

Rama	Pacientes	Sí	No	Gini
Sí (1,2)	2	2	0	$1-(1^2+0^2) = \mathbf{0.0}$
No (5,7)	2	1	1	$1-(0.5^2+0.5^2) = \mathbf{0.5}$

Gini ponderado = (2/4)×0.0 + (2/4)×0.5 = 0.250

ΔGini(Tos) = 0.375 – 0.250 = 0.125

Variable: Dolor de cabeza

Rama	Pacientes	Sí	No	Gini
Sí (1,5)	2	2	0	$1-(1^2+0^2) = \mathbf{0.0}$
No (2,7)	2	1	1	$1-(0.5^2+0.5^2) = \mathbf{0.5}$

Gini ponderado = (2/4)×0.0 + (2/4)×0.5 = 0.250

ΔGini(Dolor) = 0.375 − 0.250 = 0.125

- Se empata de nuevo: se elige **tos**

Nivel 2, rama derecha: Fiebre Baja

Distribución: Sí=1 (#8), No=3 (#3,4,6). Gini = 0.375.

#	Fiebre	Tos	Dolor de cabeza	Enfermedad
3	Baja	No	Sí	No
4	Baja	Sí	No	No
6	Baja	No	No	No
8	Baja	Sí	Sí	Sí

Variable: Tos

Rama	Pacientes	Sí	No	Gini
Sí (4,8)	2	1	1	$1-(0.5^2+0.5^2) = \mathbf{0.5}$
No (3,6)	2	0	2	$1-(0^2+1^2) = \mathbf{0.0}$

Gini ponderado = $(2/4)\times 0.5 + (2/4)\times 0.0 = 0.250$

ΔGini(Tos) $= 0.375 - 0.250 = 0.125$

Variable: Dolor de cabeza

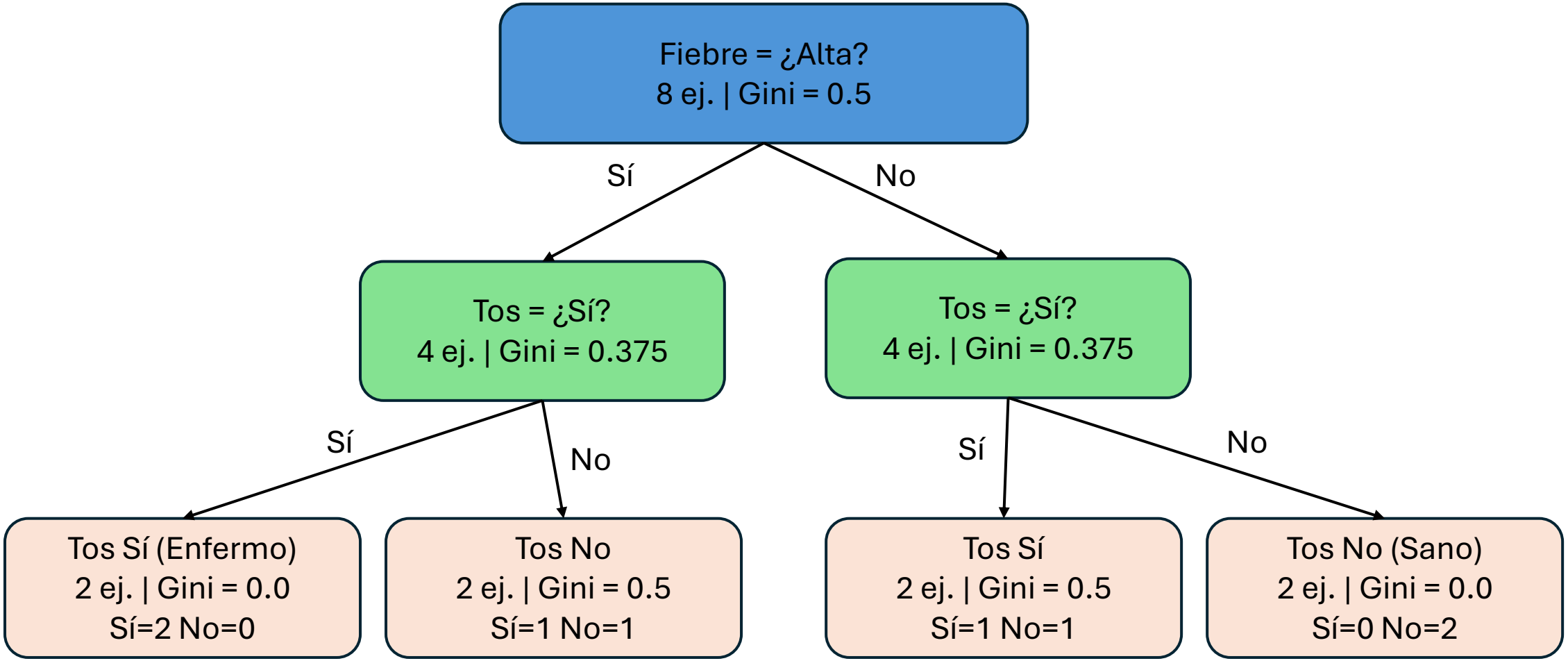
Rama	Pacientes	Sí	No	Gini
Sí (3,8)	2	1	1	$1-(0.5^2+0.5^2) = \mathbf{0.5}$
No (4,6)	2	0	2	$1-(0^2+1^2) = \mathbf{0.0}$

Gini ponderado = $(2/4)\times 0.5 + (2/4)\times 0.0 = 0.250$

ΔGini(Dolor) $= 0.375 - 0.250 = 0.125$

- Se empata de nuevo: se elige **tos**

El árbol en el segundo nivel queda:



Ya solo faltan las ultimas divisiones en el nivel 3

- **Fiebre Alta + Tos No**

#	Fiebre	Tos	Dolor de cabeza	Enfermedad
5	Alta	No	Sí	Sí
7	Alta	No	No	No

Variable: Dolor de cabeza

Rama	Pacientes	Sí	No	Gini
Sí (5)	1	1	0	$1-(1^2+0^2) = \mathbf{0.0}$
No (7)	1	0	1	$1-(0^2+1^2) = \mathbf{0.0}$

$\Delta Gini = 0.500 - 0.000 = 0.500$

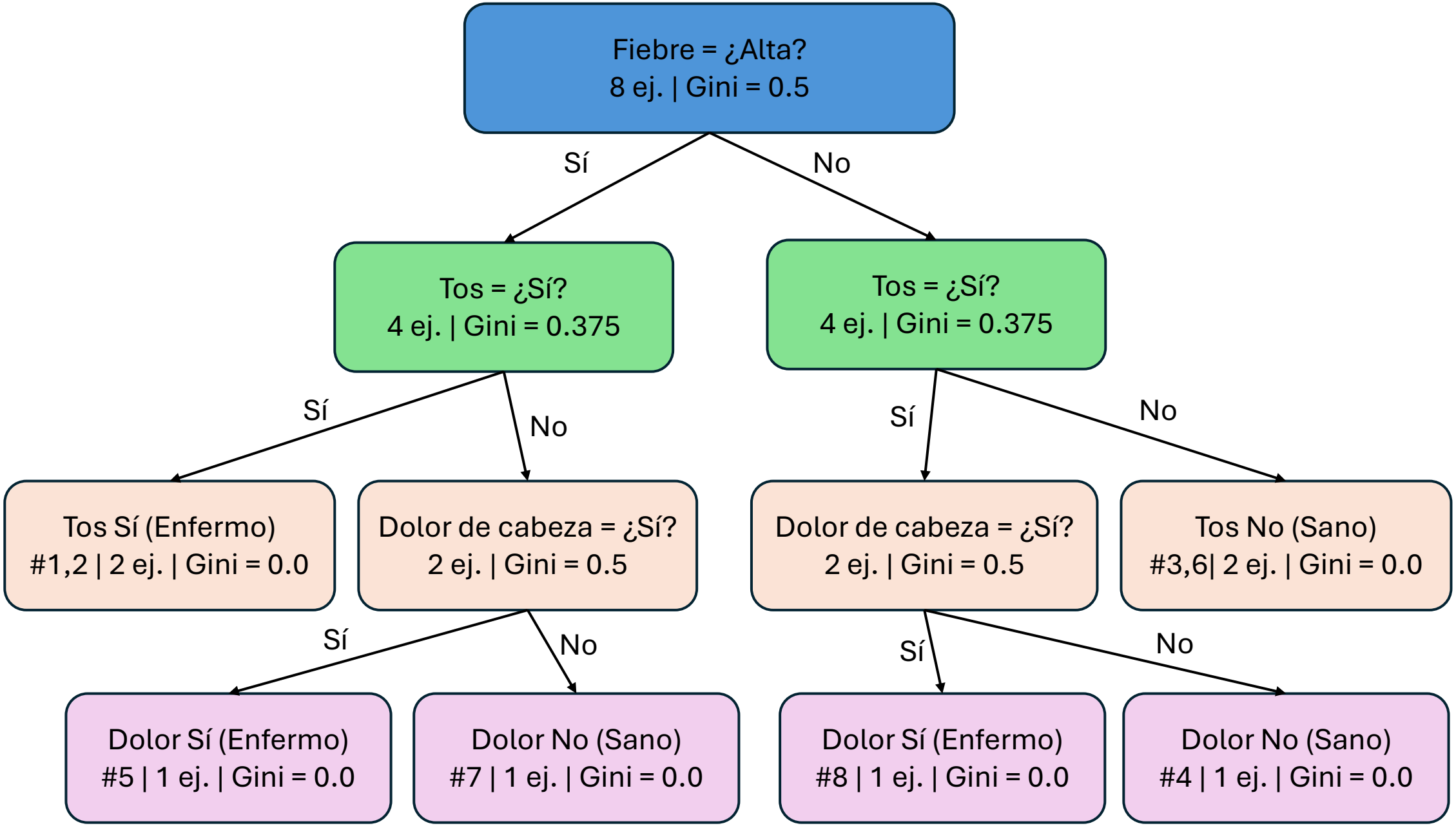
- **Fiebre Baja + Tos Sí**

#	Fiebre	Tos	Dolor de cabeza	Enfermedad
4	Baja	Sí	No	No
8	Baja	Sí	Sí	Sí

Variable: Dolor de cabeza

Rama	Pacientes	Sí	No	Gini
Sí (8)	1	1	0	$1-(1^2+0^2) = \mathbf{0.0}$
No (4)	1	0	1	$1-(0^2+1^2) = \mathbf{0.0}$

$\Delta Gini = 0.500 - 0.000 = 0.500$


Fiebre = ¿Alta?
8 ej. | Gini = 0.5
Sí
No
Tos = ¿Sí?
4 ej. | Gini = 0.375
Tos = ¿Sí?
4 ej. | Gini = 0.375
Sí
No
Sí
No
Tos Sí (Enfermo)
#1,2 | 2 ej. | Gini = 0.0
Dolor de cabeza = ¿Sí?
2 ej. | Gini = 0.5
Dolor de cabeza = ¿Sí?
2 ej. | Gini = 0.5
Tos No (Sano)
#3,6| 2 ej. | Gini = 0.0
Sí
No
Sí
No
Dolor Sí (Enfermo)
#5 | 1 ej. | Gini = 0.0
Dolor No (Sano)
#7 | 1 ej. | Gini = 0.0
Dolor Sí (Enfermo)
#8 | 1 ej. | Gini = 0.0
Dolor No (Sano)
#4 | 1 ej. | Gini = 0.0

Overfitting

- Como se puede ver, ya que no hay restricciones, CART construye un árbol que memoriza por completo los datos de entrenamiento: hay hojas que contienen exactamente un ejemplo.

- Síntomas del overfitting
  - Exactitud en entrenamiento ≈ 100%
  - Exactitud en validación/prueba es mucho menor
  - El árbol se vuelve muy profundo
  - Ramas que cubren 1-2 ejemplos

Poda (Pruning)

Pre-Pruning

Se controla durante la construcción del árbol usando hiper parámetros que van a regular la forma en que crece el árbol.

- max_depth: limita la profundidad máxima del árbol
- min_samples_split: mínimo de muestras para dividir un nodo
- min_samples_leaf: mínimo de muestras que debe tener una hoja
- min_impurity_decrease: ganancia mínima para hacer un split

Post-pruning (Cost-Complexity Pruning)

Construye el árbol completo y luego elimina las ramas que aportan poco, en otras palabras, elimina un subárbol y lo remplaza por una hoja. El algoritmo más utilizado es Cost-Complexity Pruning, el cual busca minimizar:

$$R_\alpha(T) = R(T) + \alpha|T|$$

$R(T)$: error del árbol

α : parámetro de regularización

$|T|$: numero de hojas

$\alpha = 0$, sin poda, árbol completo

$\alpha\ grande$, árbol muy simple

¿Cómo decide que podar?

Para cada nodo interno t, se calcula:

$$\alpha_t = \frac{R(t) - R(T_t)}{|\ T_t\ | - 1}$$

Donde:

$R(t)$: error si se convierte en hoja el nodo t

$R(T_t)$: error del subárbol con raíz en el nodo t

$|\ T_t\ |$: número de hojas del subárbol

α_t: alfa critico

Se poda cuando el α crítico del nodo es MENOR que el α global establecido.

- El error de clasificación del árbol se puede calcular como:

$$R(T) = \sum \frac{N_m}{N} Error(hoja)$$

$$Error(hoja) = \frac{ejemplos\ mal\ clasificados}{ejemplos\ totales\ en\ la\ hoja}$$

N_m :número de muestras en la hoja

N: número de muestras en todo el dataset

Error(hoja): proporción de ejemplos mal clasificados en esa hoja

Ejemplo

Supongamos que tenemos este subárbol:

```
    Nodo t
     /   \
Hoja1   Hoja2
```

Dataset total: N=40

Hoja 1: 10 ejemplos (8A, 2B)

Hoja 2: 10 ejemplos (7A, 3B)

Paso 1. Error del subárbol $R(T_t)$ (Error de cada hoja)

$$R(T_t) = \sum \frac{N_m}{N} Error(hoja)$$

$$R(T_t) = \left(\frac{10}{40}\right)(0.2) + \left(\frac{10}{40}\right)(0.3) = 0.125$$

Paso 2. Error si convertimos el subárbol y un nodo hoja $R(t)$

Hoja colapsada: 20 ejemplos (15A, 5B)

Predice A (mayoría)

Falla los 5 ejemplos de B

$R(t)$ = 5/40 = 0.125

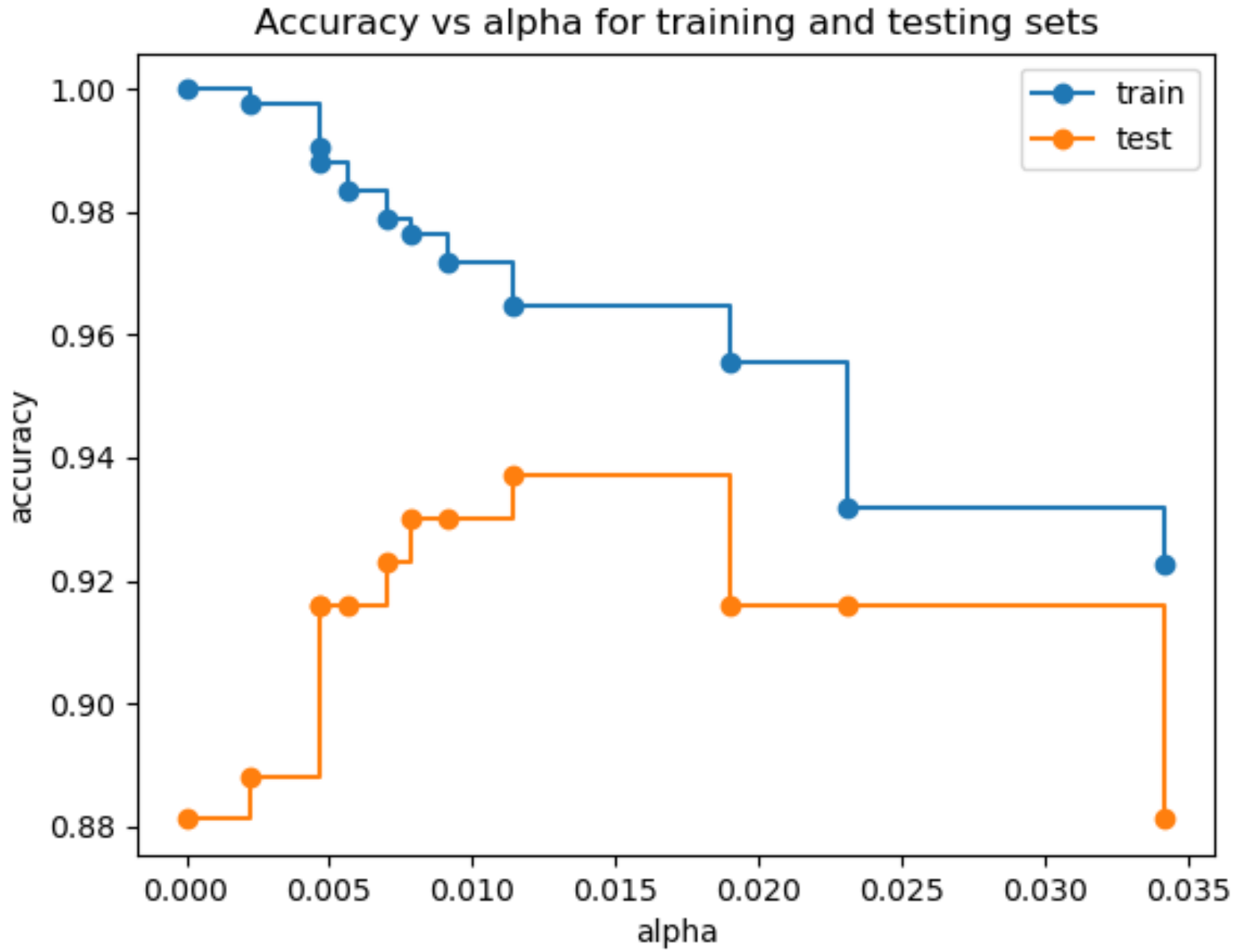
Paso 3. Calcular el alfa critico

$$\alpha_t = \frac{R(t) - R(T_t)}{|\ T_t\ | - 1} = \frac{0.125 - 0.125}{2 - 1} = \frac{0}{1} = 0$$

Si definimos el alfa global $\alpha = 0.1$ entonces, bajo este criterio ya que $\alpha_t < \alpha$, llegamos a la conclusión de que **este nodo debe de podarse.**

→ Colapsar el nodo no empeora nada

→ Mantener la rama no aporta ninguna reducción de error


Accuracy vs alpha for training and testing sets
train
test
accuracy
alpha
1.00
0.98
0.96
0.94
0.92
0.90
0.88
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035

Datos desbalanceados

- Una forma de lidiar con datos desbalanceados es con el class_weight, el cual le da a cada muestra un **peso** implícito proporcional a su clase. Esto influye en dos puntos cruciales:

1. Calculo del Gini

Las probabilidades pasan de calcularse como:

$$p_k = \frac{N_k}{N}$$

A ser probabilidades ponderadas:

$$p_k = \frac{\sum_{i \in clase\ k} w_i}{\sum_i w_i}$$

Ya no se cuentan cuantos ejemplos hay sino cuanto pesan esos ejemplos.

Supongamos un nodo con 90 ejemplos de clase A y 10 de clase B. Sin pesos las probabilidades serian:

$$p_A = 0.9, p_B = 0.1$$

Si ahora establecemos los pesos:
$w_A = 1, w_B = 10$

Entonces:

- A → 90 × 1 = 90
- B → 10 × 10 = 100

$$p_A = \frac{90}{190}, p_B = \frac{100}{190}$$

2. Asignación de la clase en la hoja
Al llegar a una hoja la clase es asignada por un conteo simple. Cuando se pesan las clases, se asigna la clase con **mayor suma de pesos**, no de mayor conteo.

$$Clase\ predicha = argmax(\sum w_i \cdot conteo(i))$$

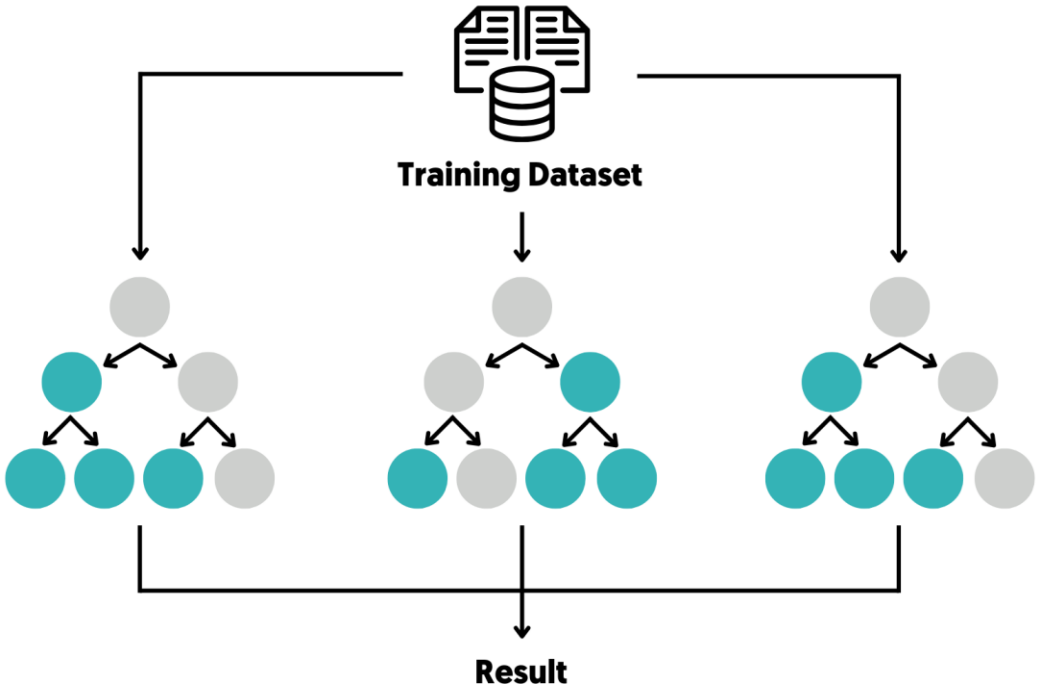
Consideraciones

- El modelo es interpretable, no requiere normalización ni escalado, puede manejar de forma nativa atributos numéricos y categóricos, no asume una distribución de los datos y es capaz de descartar atributos irrelevantes.
- Su principal problema es el sobreajuste, inestabilidad ante pequeños cambios, fronteras de decisión solo rectangulares, si hay desbalance de clases el Gini se vera afectado y el árbol tiende a predecir la clase mayoritaria.
- El max_depth es crucial para controlar la construcción del árbol, la definición de alfa nos va a definir el grado de poda del modelo, no evaluar solo el conjunto de entrenamiento.

Random Forest (Bosques aleatorios)



Random Forest



- Es un método de aprendizaje supervisado que combina muchos arboles de decisión para mejorar el rendimiento y reducir problemas como el overfitting.
- Al final, combina las predicciones de cada árbol y decide la clase por votación de la mayoría.
- Random Forest pertenece a una categoría llamada **Ensemble Learning**.

- El algoritmo trabaja con **dos técnicas** que garantizan la variedad entre los arboles.

1. **Bootstrap Aggregating (Bagging)**

Dado un conjunto de datos de tamaño N:

- Se generan múltiples subconjuntos muestreados de forma independiente, uniforme y con remplazo.
- Cada árbol del bosque se entrena con uno de esos subconjuntos.

De esta forma los arboles no se entrenan con el conjunto completo de datos, sino que a cada árbol se le entrega una muestra aleatoria en donde algunos datos se repiten y otros no aparecen.

Out Of Bag (OOB)

Considerando un conjunto de entrenamiento de tamaño n, la probabilidad de seleccionar una muestra es $\frac{1}{n}$ y la probabilidad de no seleccionarla es $1 - \frac{1}{n}$.

Probabilidad de no seleccionar una muestra n veces es:

$$\left(1 - \frac{1}{n}\right)^n$$

Si n tiende a infinito entonces:

$$\lim_{n \to \infty} \left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368$$

Lo que esto nos dice es que en promedio:

- ~63.2% de las muestras aparecen en el subconjunto
- ~36.8% quedan fuera (Out-of-Bag)

El conjunto con el que se entrena cada árbol es de **tamaño N**.

Estas muestras que no se utilizaron pueden utilizarse como un conjunto de validación interno:

Para cada dato x_i:

$$\hat{y}_i^{OOB} = \textit{agregacion de los arboles donde } x_i \textit{ fue OOB}$$

$$\text{Error } OOB = \frac{1}{N}\sum_{i=1}^{N} L\left(y_i, \hat{y}_i^{OOB}\right)$$

2. Selección aleatoria de características

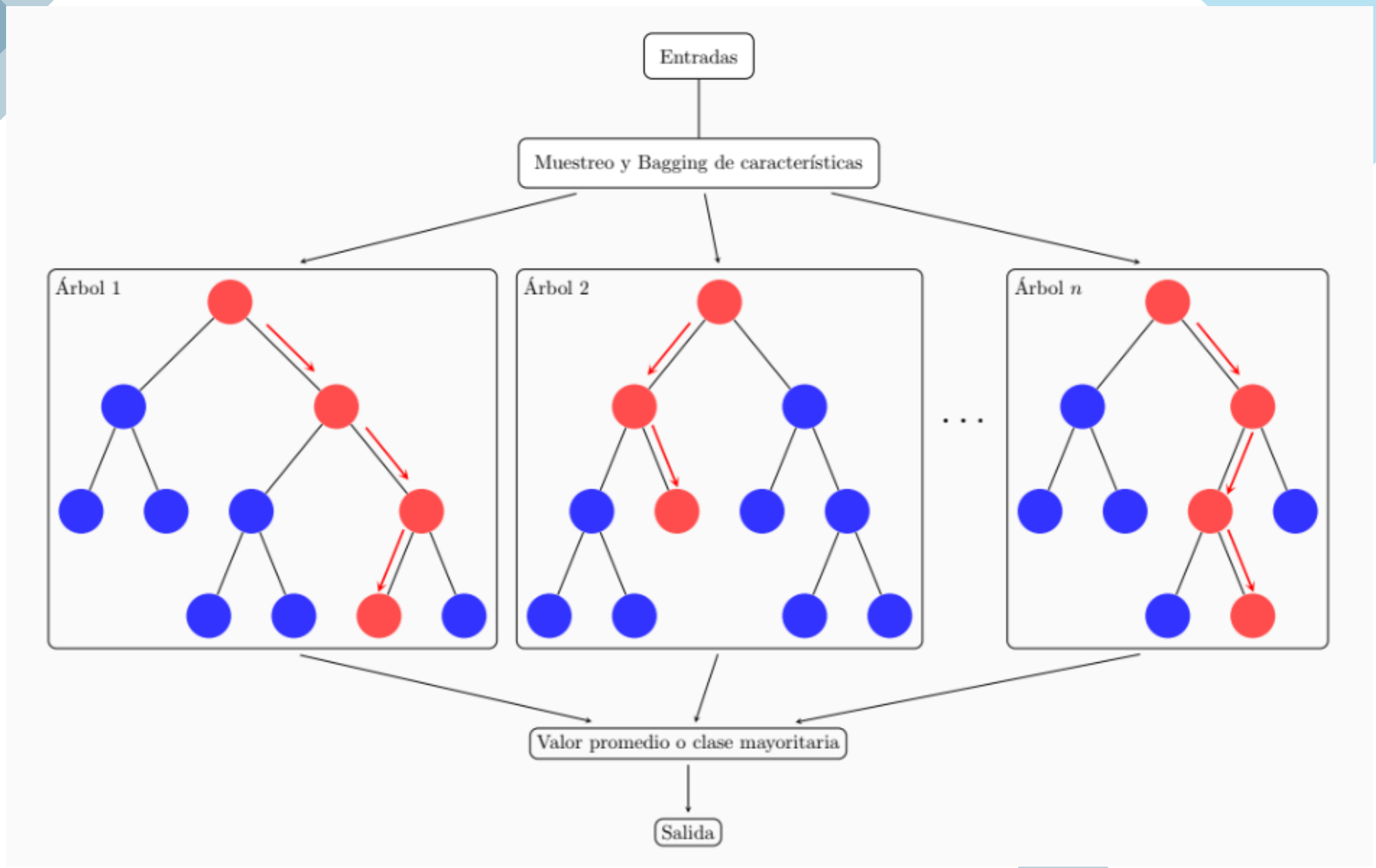
Cuando el árbol decide un split:

- No evalúa **todas** las variables
- Solo un subconjunto aleatorio de tamaño m

Esto hace que los árboles sean diversos entre sí

Votación de mayoría/promedio

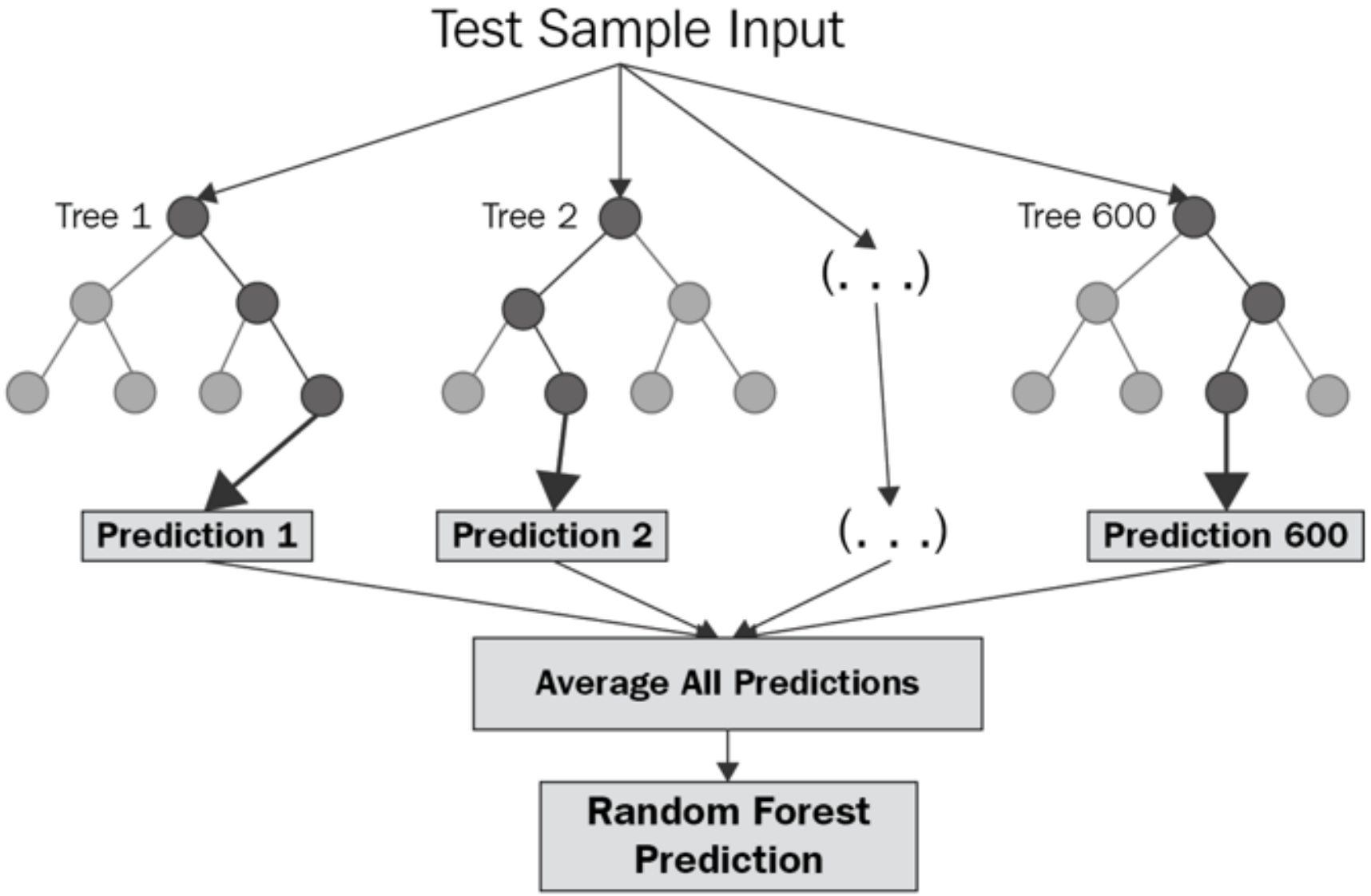
Cada árbol en el bosque vota por una clase, donde la predicción final del bosque es la clase que recibió la mayoría de votos.


Entradas
Muestreo y Bagging de características
Árbol 1
Árbol 2
Árbol n
Valor promedio o clase mayoritaria
Salida

Proceso Completo

1. Muestrear datos con reemplazo (bootstrap)
2. Construir árbol:
   - En cada nodo:
     - Seleccionar m features aleatorias
     - Elegir el mejor split entre ellas
3. Crecer el árbol (generalmente sin poda)




Test Sample Input
Tree 1
Tree 2
(. . .)
Tree 600
Prediction 1
Prediction 2
(. . .)
Prediction 600
Average All Predictions
Random Forest Prediction



¿Por qué funciona?

- Cada árbol generado tiene una alta varianza (ruidoso y se equivoca mucho), pero cada árbol vio un conjunto distinto y además utilizo distintas características.

- Esto significa que al promediar cientos de arboles que son ruidos pero independientes el error tiende a cancelarse, dejando únicamente el patrón real de los datos.

- No se hace poda a estos arboles ya que al promediar arboles que son “débiles”, el bosque entero puede volverse demasiado simple y perder precisión.

Hiper parámetros importantes

- n_estimators: número de arboles en el bosque.
- max_features: cuantas variables a considerar en cada división.
- max_depth: profundidad máxima de cada árbol.

- min_samples_split: mínimo de muestras para dividir un nodo.
- min_samples_leaf: mínimo de muestras que debe de tener un nodo.
- class_weight: peso de cada clase.

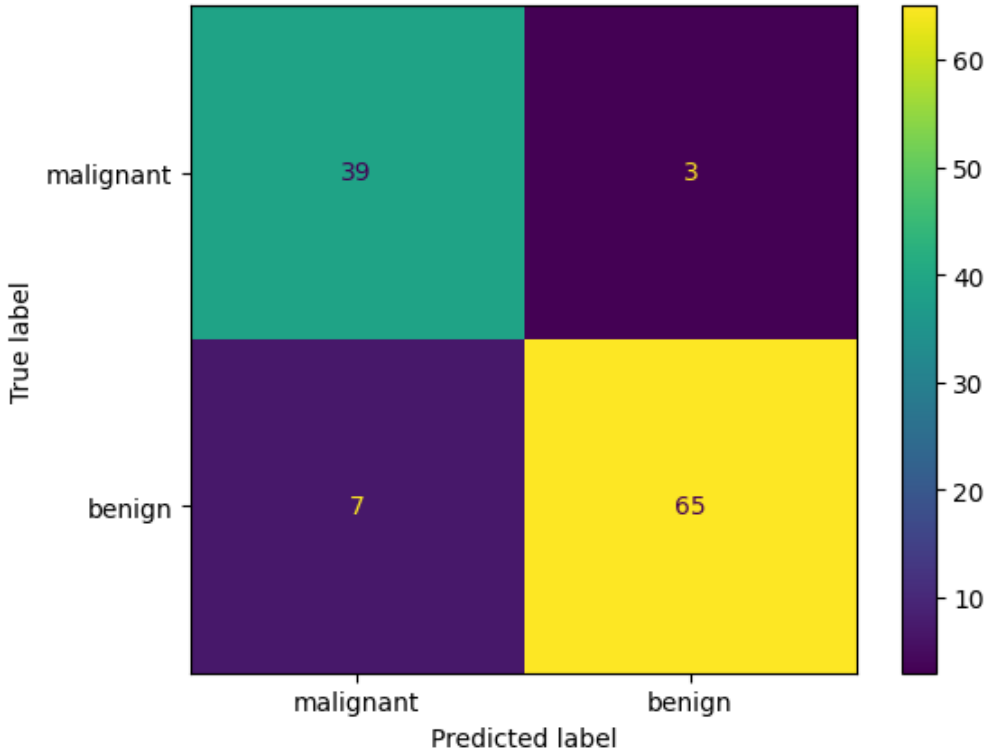
Consideraciones

- Muy buen rendimiento sin tener que ajustar demasiado los hiper parámetros, resistente al sobreajuste y al ruido, puede manejar datos no lineales, no requiere normalización.
- Poco interpretable, requiere más memoria y tiempo de procesamiento para entrenar y predecir que un solo árbol, además de que no siempre mejorara con más arboles.
- Pude manejar clases altamente desbalanceadas, las fronteras de decisión pueden no ser las mejores en ciertos casos, si hay características muy correlacionas el modelo pierde eficiencia.

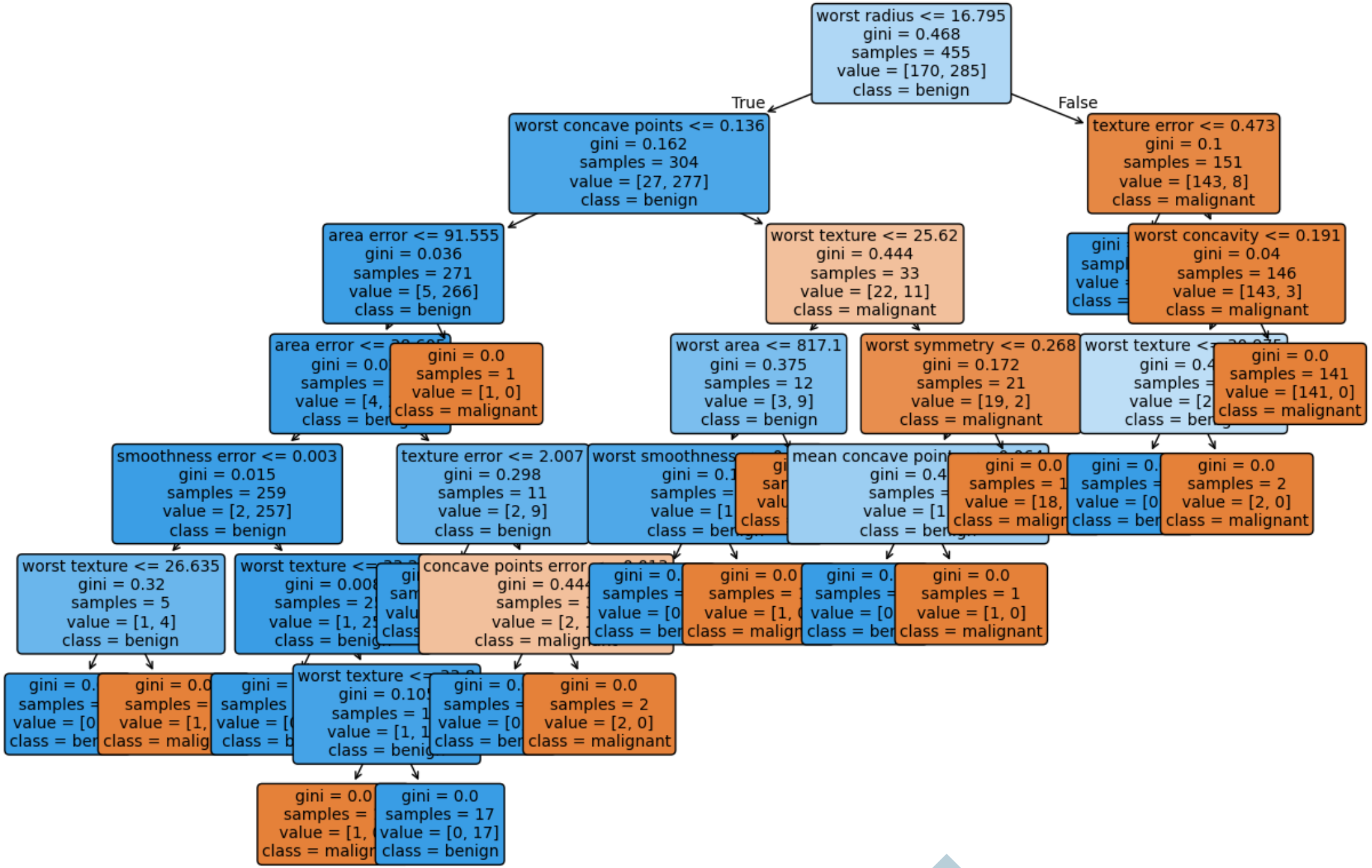
CART vs RF

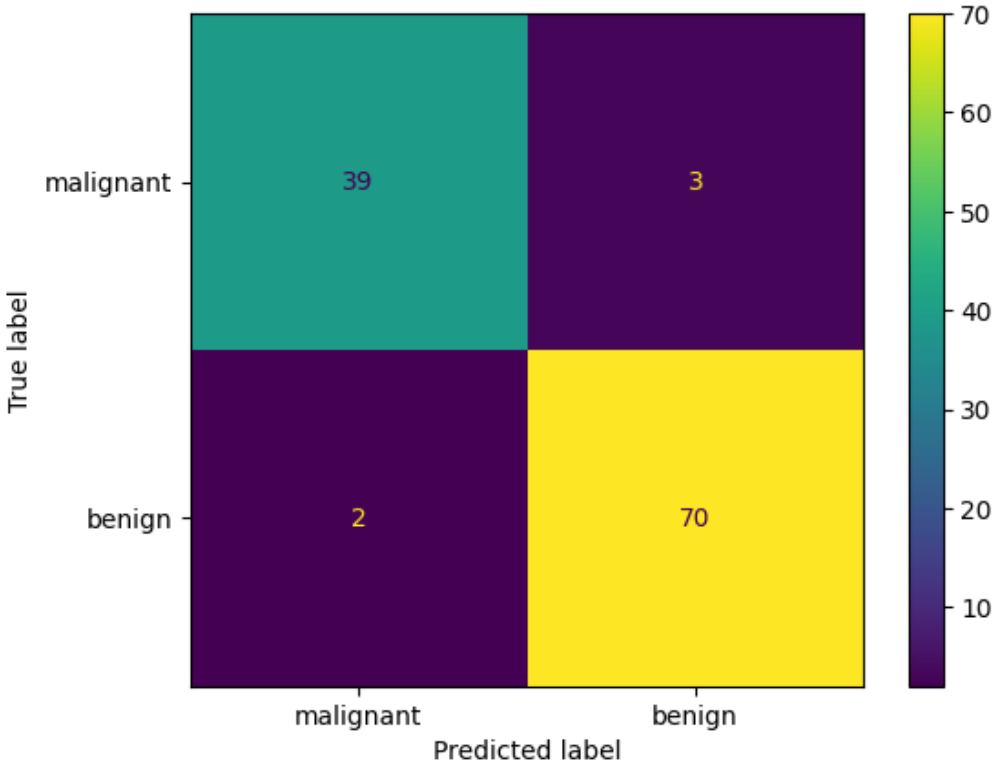
- CART

Exactitud: 0.9122



	Clase 0	**Clase 1**
Precisión	0.8479	0.9558
Recall	0.9285	0.9027
F1-Score	0.8863	0.9285


worst radius <= 16.795
gini = 0.468
samples = 455
value = [170, 285]
class = benign
True
False
worst concave points <= 0.136
gini = 0.162
samples = 304
value = [27, 277]
class = benign
texture error <= 0.473
gini = 0.1
samples = 151
value = [143, 8]
class = malignant
area error <= 91.555
gini = 0.036
samples = 271
value = [5, 266]
class = benign
worst texture <= 25.62
gini = 0.444
samples = 33
value = [22, 11]
class = malignant
worst concavity <= 0.191
gini = 0.04
samples = 146
value = [143, 3]
class = malignant
gini = 0.0
samples = 1
value = [1, 0]
class = malignant
worst area <= 817.1
gini = 0.375
samples = 12
value = [3, 9]
class = benign
worst symmetry <= 0.268
gini = 0.172
samples = 21
value = [19, 2]
class = malignant
gini = 0.0
samples = 141
value = [141, 0]
class = malignant
smoothness error <= 0.003
gini = 0.015
samples = 259
value = [2, 257]
class = benign
texture error <= 2.007
gini = 0.298
samples = 11
value = [2, 9]
class = benign
gini = 0.0
samples = 2
value = [2, 0]
class = malignant
worst texture <= 26.635
gini = 0.32
samples = 5
value = [1, 4]
class = benign
gini = 0.0
samples = 1
value = [1, 0]
class = malignant
gini = 0.0
samples = 17
value = [0, 17]
class = benign



- RF

Exactitud: 0.9561

	Clase 0	**Clase 1**
Precisión	0.9512	0.9589
Recall	0.9285	0.9722
F1-Score	0.9397	0.9655

Cross Validation
CART mean accuracy: 0.9173
Random Forest mean accuracy: 0.9543

Para la próxima clase...

Boosting

